

Lab Notebook #8:

Mitigating computational bias using Fairlearn

Fairness in AI is a multidisciplinary field focused on understanding and reducing the potential negative impacts of AI on society. In this tutorial, we'll evaluate and address fairness-related issues within the context of the U.S. healthcare system. Using the Python library Fairlearn, we'll examine machine learning models for performance differences across sensitive groups and explore different algorithmic techniques to help reduce these disparities.

Install packages

```
!pip install Fairlearn
!pip install scikit-learn
```

```
Requirement already satisfied: Fairlearn in /usr/local/lib/python3.12/dist-packages (0.13.0)
Requirement already satisfied: narwhals>=1.14.0 in /usr/local/lib/python3.12/dist-packages (from Fairlearn) (2.8.0)
Requirement already satisfied: numpy>=1.24.4 in /usr/local/lib/python3.12/dist-packages (from Fairlearn) (2.0.2)
Requirement already satisfied: pandas>=2.0.3 in /usr/local/lib/python3.12/dist-packages (from Fairlearn) (2.2.2)
Requirement already satisfied: scikit-learn>=1.2.1 in /usr/local/lib/python3.12/dist-packages (from Fairlearn) (1.6.1)
Requirement already satisfied: scipy<1.16.0,>=1.9.3 in /usr/local/lib/python3.12/dist-packages (from Fairlearn) (1.15.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=2.0.3->Fairlearn)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=2.0.3->Fairlearn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=2.0.3->Fairlearn) (2025.2)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.2.1->Fairlearn) (1.
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.2.1->Fairle
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=2.0.3->
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.15.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
```

Import and set up packages

```
import numpy as np
import pandas as pd

pd.set_option("display.float_format", "{:.3f}".format)
```

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set()
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.utils import Bunch
from sklearn.metrics import (
    balanced_accuracy_score,
    roc_auc_score,
    accuracy_score,
    recall_score,
    confusion_matrix,
    roc_auc_score,
    roc_curve,
    RocCurveDisplay)
from sklearn import set_config

set_config(display="diagram")
```

```
from fairlearn.metrics import (
    MetricFrame,
    true_positive_rate,
    false_positive_rate,
    false_negative_rate,
    selection_rate,
    count,
    false_negative_rate_difference
)
```

```
from fairlearn.postprocessing import ThresholdOptimizer, plot_threshold_optimizer
from fairlearn.postprocessing._interpolated_thresholder import InterpolatedThresholder
from fairlearn.postprocessing._threshold_operation import ThresholdOperation
from fairlearn.reductions import ExponentiatedGradient, EqualizedOdds, TruePositiveRateParity
```

```
from IPython import display
from datetime import date
```

Introduction of Fairlearn and other tutorial resources

[Fairlearn](#)

Fairlearn is an open-source, community-driven project to help data scientists improve fairness of AI systems. It includes:

- A Python library for fairness assessment and improvement (fairness metrics, mitigation algorithms, plotting, etc.)
- Educational resources covering organizational and technical processes for unfairness mitigation (user guide, case studies, Jupyter notebooks, etc.)

The project was started in 2018 at Microsoft Research. In 2021 it adopted a neutral governance structure and since then it is completely community-driven.

Introduction to the health care scenario

The following scenario builds on previous research that highlights racial disparities in how health care resources are allocated in the U.S. ([Obermeyer et al., 2019](#)). Motivated by that work, in this tutorial we consider an automated system for recommending patients for *high-risk care management* programs, which are described by Obermeyer et al. 2019 as follows:

These programs seek to improve the care of patients with complex health needs by providing additional resources, including greater attention from trained providers, to help ensure that care is well coordinated. Most health systems use these programs as the cornerstone of population health management efforts, and they are widely considered effective at improving outcomes and satisfaction while reducing costs. [...] Because the programs are themselves expensive—with costs going toward teams of dedicated nurses, extra primary care appointment slots, and other scarce resources—**health systems rely extensively on algorithms to identify patients who will benefit the most.**

Convenience restriction

- In practice, the modeling of health needs would use large data sets covering a wide range of diagnoses. In this tutorial, we will work with a [publicly available clinical dataset](#) that focuses on *diabetes patients only* ([Strack et al., 2014](#)).

Dataset and task

We'll be working with a clinical dataset that tracks hospital readmissions for diabetes patients over a ten-year period (1998–2008) from 130 hospitals in the U.S. Each record reflects details of a hospital stay for a patient with diabetes, lasting between one and fourteen days.

This dataset includes information on patient demographics, diagnoses, diabetes medications, number of prior hospital visits, payer details, and whether a patient was readmitted within 30 days after release.

Our goal is to build a classification model that predicts whether patients should be referred to their primary care physicians for a high-risk care management program. A positive prediction would mean a recommendation for enrollment in this program.

Decision Point: Task Definition

We'll use readmission within 30 days as a **proxy** for identifying patients who might need more support after discharge. This will serve as our prediction target.

Due to the class imbalance, we'll use balanced accuracy to measure model performance. Another key metric will be selection rate, indicating the proportion of patients recommended for care.

In an ideal scenario, healthcare professionals would help design and validate the model to ensure the task definition aligns closely with patient care needs.

Fairness considerations

To evaluate and address potential harms, especially those that may impact specific groups disproportionately, here are some key considerations:

- Which groups are most likely to be disproportionately negatively affected? Prior studies indicate that disparities can exist across racial and ethnic groups. Therefore, we should pay special attention to how the model performs across these groups to ensure fair access to resources and support.
- What are the harms? The primary concern here is allocation harm. Specifically, false negatives pose a risk, as they mean that patients who are likely to be readmitted are not recommended for the high-risk care management program. This could result in those needing support not receiving it, potentially affecting their health outcomes.
- How should we measure these harms? We can quantify these harms by monitoring false negative rates for each group, comparing disparities between them. Additionally, using metrics like equalized odds (which ensures similar error rates across groups) and demographic parity (which ensures a similar selection rate across groups) can provide a clearer picture of whether allocation harms are occurring, and if so, to what extent.

In the remainder of the tutorial we will:

- Examine the dataset and our choice of label with an eye towards a variety of fairness issues.
- Then, train a logistic regression model and assess its performance as well as fairness.
- Finally, look at two unfairness mitigation strategies.

✓ Task definition and dataset characteristics

In this section, we'll start by loading the dataset and examining it for potential fairness issues. Key areas to focus on include:

- Sample Sizes Across Demographic Groups: We'll first check the sample sizes of various demographic groups, paying particular attention to different racial groups. This helps ensure that all groups are adequately represented, as imbalances could impact model performance and fairness.
- Choice of label appropriateness: Next, we'll consider whether our chosen label — readmission within 30 days — is appropriate for our task. It's essential that the label accurately reflects the outcome we aim to predict to ensure our model's effectiveness and fairness.

In addition to dataset characteristics, it's important to consider how the data was collected. Ensuring that data collection respects the autonomy of individuals is a key aspect of ethical AI practices, as it aligns with transparency and fairness principles.

To help us document and communicate our understanding of the dataset, we'll use the datasheets method. By systematically documenting dataset characteristics and any potential concerns, we can anticipate and highlight issues that may arise during model training, evaluation, and deployment.

✓ Load the dataset

We can now load the dataset and review the meaning of its columns.

```
df = pd.read_csv("https://respa1.s3.us-east-2.amazonaws.com/fairlearn_data_preprocessed.csv")
```

```
df.head()
```

	race	gender	age	discharge_disposition_id	admission_source_id	time_in_hospital	medical_specialty	num_lab_pr
0	Caucasian	Female	30 years or younger	Other	Referral	1	Other	
1	Caucasian	Female	30 years or younger	Discharged to Home	Emergency	3	Missing	
2	AfricanAmerican	Female	30 years or younger	Discharged to Home	Emergency	2	Missing	
3	Caucasian	Male	30-60 years	Discharged to Home	Emergency	2	Missing	
4	Caucasian	Male	30-60 years	Discharged to Home	Emergency	1	Missing	

5 rows × 25 columns

df.shape

(101766, 25)

The columns contain mostly boolean and categorical data (including age and various test results), with just the following exceptions:

`time_in_hospital`, `num_lab_procedures`, `num_procedures`, `num_medications`, `number_diagnoses`.

features	description
race, gender, age	demographic features
medicare, medicaid	insurance information
admission_source_id	emergency, referral, or other
had_emergency, had_inpatient_days, had_outpatient_days	hospital visits in prior year
medical_specialty	admitting physician's specialty
time_in_hospital, num_lab_procedures, num_procedures, num_medications, primary_diagnosis, number_diagnoses, max_glu_serum, A1Cresult, insulin change, diabetesMed	description of the hospital visit
discharge_disposition_id	discharged to home or not
readmitted, readmit_binary, readmit_30_days	readmission information

```
# Show the values of all binary and categorical features
categorical_values = {}
for col in df:
    if col not in {'time_in_hospital', 'num_lab_procedures',
                  'num_procedures', 'num_medications', 'number_diagnoses'}:
        categorical_values[col] = pd.Series(df[col].value_counts().index.values)
categorical_values_df = pd.DataFrame(categorical_values).fillna('')
categorical_values_df.T
```

	0	1	2	3	4	5
race	Caucasian	AfricanAmerican	Unknown	Hispanic	Other	Asian
gender	Female	Male	Unknown/Invalid			
age	Over 60 years	30-60 years	30 years or younger			
discharge_disposition_id	Discharged to Home	Other				
admission_source_id	Emergency	Referral	Other			
medical_specialty	Missing	Other	InternalMedicine	Emergency/Trauma	Family/GeneralPractice	Cardiology
primary_diagnosis	Other	Respiratory Issues	Diabetes	Genitourinary Issues	Musculoskeletal Issues	
max_glu_serum	Norm	>200	>300			
A1Cresult	>8	Norm	>7			
insulin	No	Steady	Down	Up		
change	No	Ch				
diabetesMed	Yes	No				
medicare	False	True				
medicaid	False	True				
had_emergency	False	True				
had_inpatient_days	False	True				
had_outpatient_days	False	True				
readmitted	NO	>30	<30			
readmit_binary	0.000	1.000				

We mark all categorical features:

```
categorical_features = [
    "race",
    "gender",
    "age",
    "discharge_disposition_id",
    "admission_source_id",
    "medical_specialty",
    "primary_diagnosis",
    "max_glu_serum",
    "A1Cresult",
```

```
"insulin",  
"change",  
"diabetesMed",  
"readmitted"  
]
```

```
for col_name in categorical_features:  
    df[col_name] = df[col_name].astype("category")
```

✓ Group sample sizes

In fairness assessment, an essential characteristic to consider is the sample size of the demographic groups we're assessing. Smaller sample sizes can impact both the assessment and model training stages:

- **Assessment:** With smaller groups, it becomes harder to assess the impact of the AI system because the limited data points increase uncertainty (or error) in our estimates. This larger margin of error can make it difficult to draw reliable conclusions about fairness impacts on these groups.
- **Model Training:** Smaller sample sizes also mean fewer data points available during model training, making it challenging for the model to capture specific patterns for these groups. As a result, the model's predictive performance may be worse for these groups, potentially leading to biased outcomes.

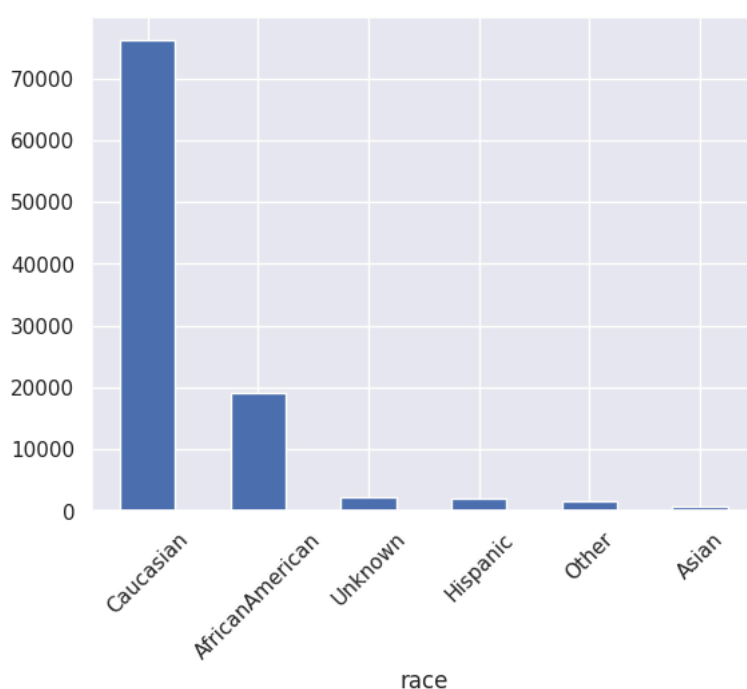
Let's start by examining the sample sizes for the groups in our dataset according to the race variable. This will give us insight into how well each group is represented, which is crucial for understanding any fairness concerns.

```
df["race"].value_counts()
```

count	
race	
Caucasian	76099
AfricanAmerican	19210
Unknown	2273
Hispanic	2037
Other	1506
Asian	641

dtype: int64

```
df["race"].value_counts().plot(kind='bar', rot=45);
```



Normalized as frequencies:

```
df["race"].value_counts(normalize=True)
```

proportion	
race	
Caucasian	0.748
AfricanAmerican	0.189
Unknown	0.022
Hispanic	0.020
Other	0.015
Asian	0.006

dtype: float64

In our dataset, our patients are predominantly *Caucasian* (75%). The next largest racial group is *AfricanAmerican*, making up 19% of the patients. The remaining race categories (including *Unknown*) compose only 6% of the data.

✓ YOUR TASK: Examine the dataset composition by **gender**:

```
# counts
df["gender"].value_counts()
```

count	
gender	
Female	54708
Male	47055
Unknown/Invalid	3

dtype: int64

```
# frequencies
df["gender"].value_counts(normalize=True)
```

proportion	
gender	
Female	0.538
Male	0.462
Unknown/Invalid	0.000

dtype: float64

✓ YOUR TASK: Write a brief comment to describe the distribution of the data by gender

```
# Comment:
```

```
#The distribution is quite balanced with females being slightly more in number. There are just 3 unknown cases which is fairly small.
```

✓ Decision point: How do we address smaller group sizes?

When certain groups are underrepresented in the dataset, it limits our ability to effectively assess fairness. Here are three possible interventions we can consider (and even combine) to address this:

- **Collect More Data:** Gather additional data for underrepresented groups. This is the most reliable approach for improving assessment and performance but may not always be feasible.
- **Bucketing Groups:** Combine smaller groups with similar characteristics to increase sample size. While this approach can improve the reliability of assessments, we need to ensure that the combined groups share similar risks. Otherwise, merging could mask potential harms.
- **Dropping Small Groups:** Remove groups with very few samples if they are too small to support meaningful analysis. However, dropping groups can lead to representational harm by erasing them from the analysis. Therefore, this approach is recommended only when absolutely necessary.

In all cases, decisions to merge or drop groups should be clearly documented in the datasheet to ensure transparency and traceability.

For our dataset, we'll use the following strategies:

- **Merging Groups:** We'll merge the three smallest racial groups (Asian, Hispanic, and Other) to create a combined category. However, we'll retain the original group labels for auxiliary assessments, in case we need them for more nuanced analysis.
- **Dropping Gender Group:** The gender group labeled Unknown/Invalid has such a small sample size that assessing fairness meaningfully isn't feasible. We'll remove this group but document the decision in our datasheet to avoid representational harm wherever possible.

```
# retain the original race as race_all, and merge Asian+Hispanic+Other
df["race_all"] = df["race"]
df["race"] = df["race"].replace({"Asian": "Other", "Hispanic": "Other"})

# drop gender group Unknown/Invalid
df = df.query("gender != 'Unknown/Invalid'")
```

```
/tmp/ipython-input-883845803.py:3: FutureWarning: The behavior of Series.replace (and DataFrame.replace) with CategoricalDtype is
df["race"] = df["race"].replace({"Asian": "Other", "Hispanic": "Other"})
```

✓ Examining the choice of label

Next we dive into the question of whether our choice of label (readmission within 30 days) aligns with our goal (identify patients that would benefit from the care management program).

A framework particularly suited for this analysis is called *measurement modeling* (see, e.g., [Jacobs and Wallach, 2021](#)). The goal of measurement modeling is to describe the relationship between what we care about and what we can measure. The thing that we care about is referred to as the *construct* and what we can observe is referred to as the *measurement*. In our case:

- **construct** = greatest benefit from the care management program
- **measurement** = readmission within 30 days (in the absence of such program)

In our case, the **measurement** coincides with the **classification label**.

The act of *operationalizing* the construct via a specific measurement corresponds to making certain assumptions. In our case, we are making the following assumption: **the greatest benefit from the care management program would go to patients that are** (in the absence of such a program) **most likely to be readmitted into the hospital within 30 days**.

✓ How can we check whether our assumptions apply?

In the terminology of measurement modeling, how do we establish *construct validity*? Following, [Jacobs and Wallach, 2021](#),

Establishing construct validity means demonstrating, in a variety of ways, that the measurements obtained from measurement model are both meaningful and useful:

- Does the operationalization capture all relevant aspects of the construct purported to be measured?
 - Do the measurements look plausible?
 - Do they correlate with other measurements of the same construct? Or do they vary in ways that suggest that the operationalization may be inadvertently capturing aspects of other constructs?
 - Are the measurements predictive of measurements of any relevant observable properties (and other unobservable theoretical constructs) thought to be related to the construct, but not incorporated into the operationalization?

We focus on one aspect of construct validity, called *predictive validity*, which refers to the extent to which the measurements obtained from a measurement model are predictive of measurements of any relevant observable properties related to the construct purported to be measured, but not incorporated into the operationalization.

✓ Predictive validity

We need to show that our measurement `readmit_30_days` is correlated with patient characteristics that are related to our construct "benefiting from care management". One such characteristic is the general patient health, where we expect that patients that are less

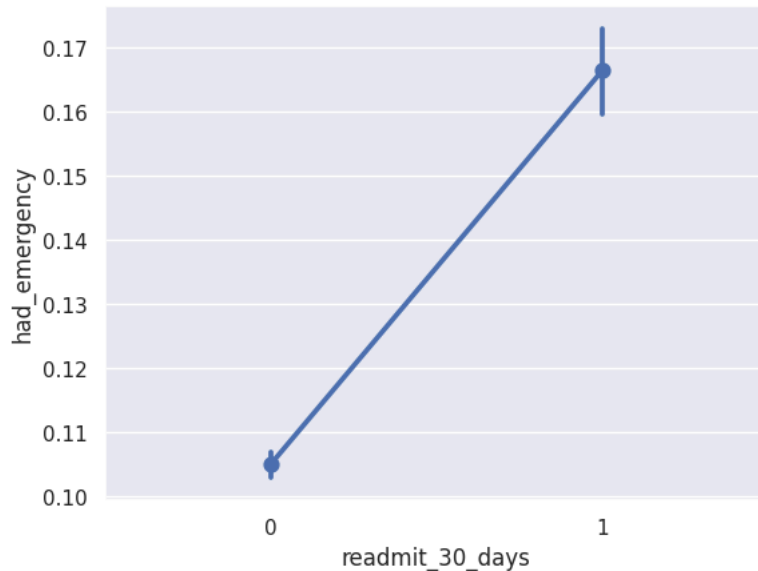
healthy are more likely to benefit from care management.

While our data does not contain full health records that would enable us to holistically measure general patient health, the data does contain two relevant features: `had_emergency` and `had_inpatient_days`, which indicate whether the patient spent any days in the emergency room or in the hospital (but non-emergency) in the preceding year.

To establish predictive validity, we would like to show that our measurement `readmit_30_days` is predictive of these two observable characteristics.

First, let's check the rate at which the patients with different `readmit_30_days` labels were readmitted in the previous year:

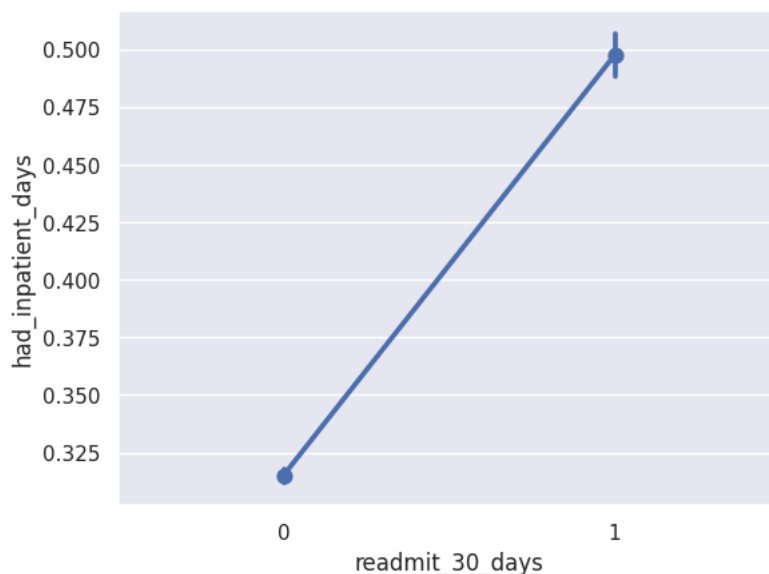
```
sns.pointplot(y="had_emergency", x="readmit_30_days",
              data=df, errorbar=('ci', 95));
```



The plot shows that patients with `readmit_30_days=0` have a lower rate of emergency visits in the prior year, whereas patients with `readmit_30_days=1` have a larger rate. (The vertical lines indicate 95% confidence intervals obtained via bootstrapping.)

We see a similar pattern when `readmit_30_days` is used to predict the rate of (non-emergency) hospital visits in the previous year:

```
sns.pointplot(y="had_inpatient_days", x="readmit_30_days",
              data=df, errorbar=('ci', 95));
```

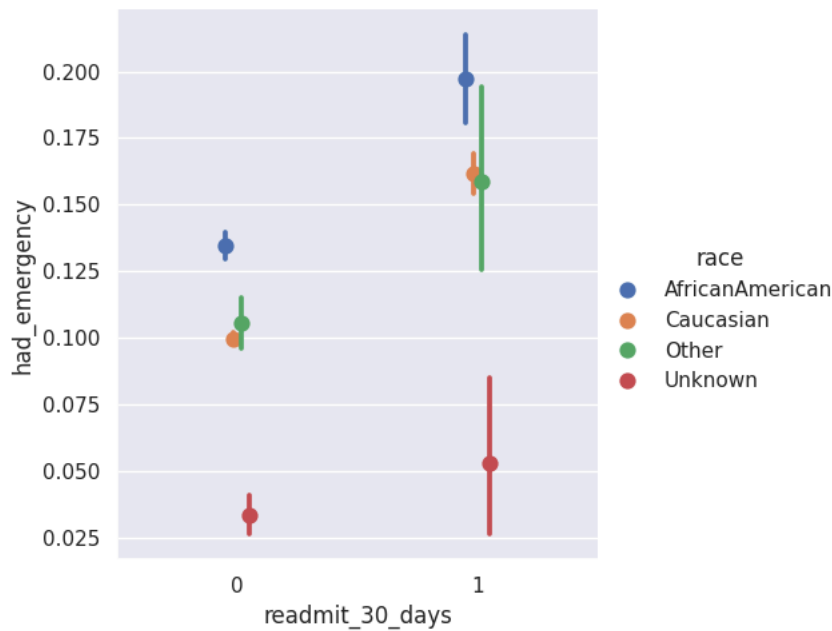


Now let's take a look whether the predictiveness is similar across different race groups. First, let's check how well `readmit_30_days` predicts `had_emergency`:

```
# Visualize predictiveness using a categorical pointplot
sns.catplot(y="had_emergency", x="readmit_30_days", hue="race", data=df,
```



```
kind="point", dodge=True, errorbar=('ci', 95), linestyle='none');
```

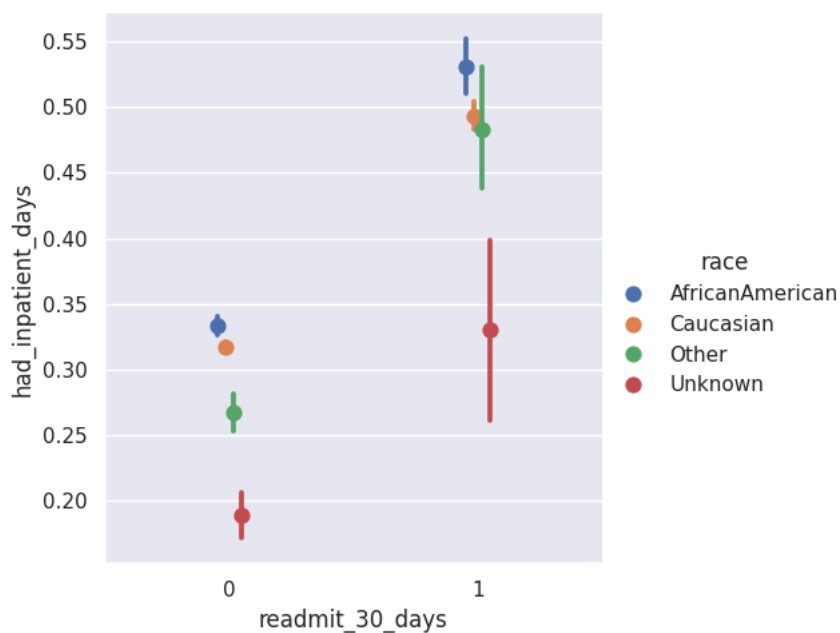


The patients in the group *Unknown* have a substantially lower rate of emergency visits in the prior year, regardless of whether they are readmitted in 30 days. The readmission is still positively correlated with `had_emergency`, but note the large error bars (due to small sample sizes).

We also see that the group with feature value *AfricanAmerican* has a higher rate of emergency visits compared with other groups. However, generally the groups *Caucasian*, *AfricanAmerican* and *Other* follow similar dependence patterns.

YOUR TASK: Visualize the predictive validity for `readmit_30_days` in predicting the rate of (non-emergency) hospital visits in the previous year:

```
#Hint: Use the had_inpatient_days variable
sns.catplot(y="had_inpatient_days", x="readmit_30_days", hue="race", data=df,
            kind="point", dodge=True, errorbar=('ci', 95), linestyle='none');
```



Again, for *Unknown* the rate of (non-emergency) hospital visits in the previous year is lower than for other groups. In all groups there is a strong positive correlation between `readmit_30_days` and `had_inpatient_days`.

In all cases, we see that readmission in 30 days is predictive of our two measurements of general patient health.

The analysis is also surfacing the fact that patients with the value of race *Unknown* have fewer hospital visits in the preceding year (both emergency and non-emergency) than other groups. In practice, this would be a good reason to reach out to health professionals to

investigate this patient cohort, to make sure that we understand why there is the systematic difference.

✓ Label imbalance

Now that we have established the validity of our label, we will check frequency of its values in our data. The frequency of different labels is an important descriptive characteristic in classification settings for several reasons:

- some classification algorithms and performance measures might not work well with data sets with extreme class imbalance
- in binary classification settings, our ability to evaluate error is often driven by the size of the smaller of the two classes (again, the smaller the sample the larger the uncertainty in estimates)
- label imbalance may exacerbate the problems due to smaller group sizes in fairness assessment

Let's check how many samples in our data are labeled as positive and how many as negative.

```
df["readmit_30_days"].value_counts() # counts
```

	count
readmit_30_days	
0	90406
1	11357

dtype: int64

```
df["readmit_30_days"].value_counts(normalize=True) # frequencies
```

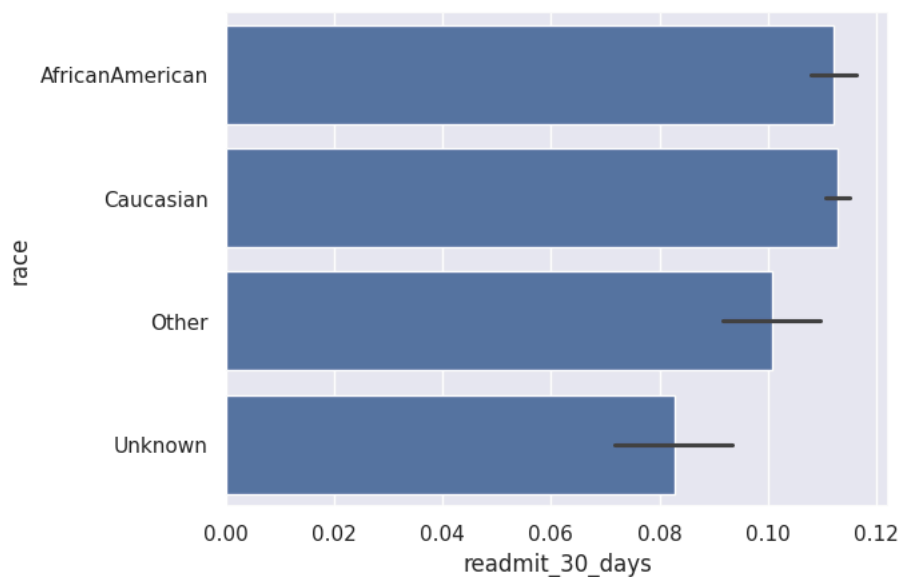
	proportion
readmit_30_days	
0	0.888
1	0.112

dtype: float64

As we can see, the target label is heavily skewed towards the patients not being readmitted within 30 days. In our dataset, only 11% of patients were readmitted within 30 days.

Now, let's examine how much the label frequencies vary within each group defined by `race`:

```
sns.barplot(x="readmit_30_days", y="race", data=df, errorbar=('ci', 95));
```



We see the rate of 30-day readmission is similar for the *AfricanAmerican* and *Caucasian* groups, but appears smaller for *Other* and smallest for *Unknown* (this is consistent with an overall lower rate of hospital visits in the prior year). The smaller sample size of the *Other* and *Unknown* groups mean that there is more uncertainty around the estimate for these two groups.

Proxies for sensitive features

Next, we'll look into identifying features that are highly predictive of the sensitive feature race; these features are known as proxies.

In this tutorial, we primarily address fairness by examining the impact of the machine-learning model across different groups. However, other fairness perspectives focus on how the model may utilize information from sensitive features and which uses of this information are justifiable — often analyzed through causal reasoning. Additionally, certain uses of sensitive features (or their proxies) may be restricted by law in some settings.

Understanding proxies is also important because they can help explain observed impact differences among groups, even if the model doesn't explicitly use the sensitive feature.

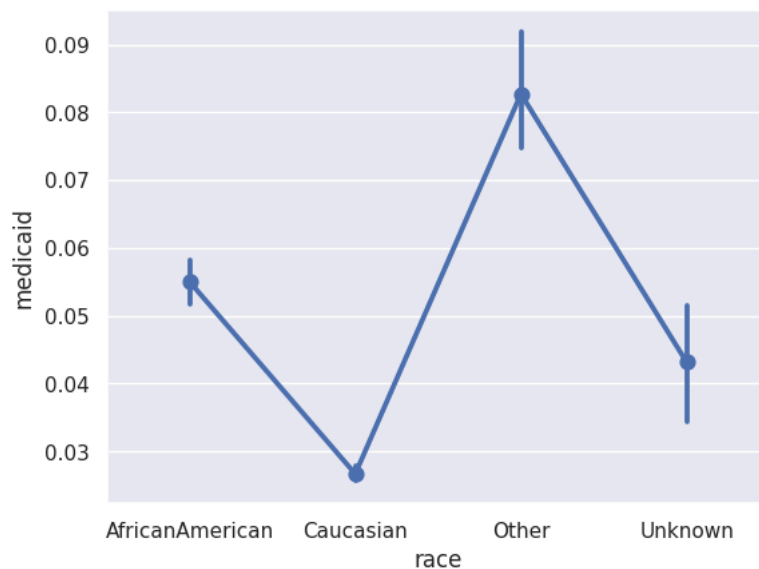
In this section, we'll touch on identifying such proxies, without delving into the legal or causal aspects.

In the United States, Medicare and Medicaid are federal programs designed to help eligible people cover healthcare costs. Medicare is generally available for people over 65 and younger people with serious health conditions, while Medicaid assists low-income people under 65 whose adjusted gross income is below the Federal Poverty Line.

Let's start by examining how Medicaid as a payment source relates to our demographic features. Since socioeconomic status often correlates with race in the United States, we can expect a relationship between race and payment with Medicaid.

```
sns.pointplot(y="medicaid", x="race", data=df)
```

<Axes: xlabel='race', ylabel='medicaid'>

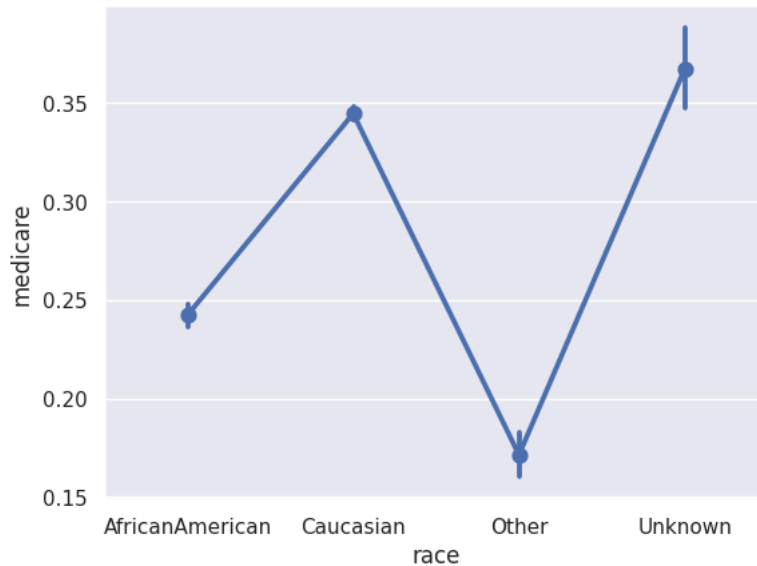


Our analysis confirms that there is indeed a relationship between paying with Medicaid and patient race. Caucasian patients are less likely to pay with Medicaid compared to other groups. Since Medicaid often serves as a proxy for socioeconomic status, these patterns align with known socioeconomic disparities across racial groups in the United States, reflecting broader patterns of wealth distribution and access to resources.

YOUR TASK: Plot the relationship between race and Medicare and add a brief comment describing what you observe.

```
sns.pointplot(y="medicare", x="race", data=df)
```

<Axes: xlabel='race', ylabel='medicare'>



Comment:

Unknown has the highest Average of paying by Medicare while the others are lower. Other group has the lowest average comparatively.

✓ Training the initial model

To choose a model for predicting 30-day readmissions, it's essential to consider the trade-offs between interpretability and model expressiveness. Here's a breakdown of the options:

- **Interpretability:** This factor is critical for fairness, as interpretable models enable stakeholders to understand and scrutinize predictions. Interpretability matters because:
 - Stakeholders might identify fairness issues that data scientists could overlook.
 - A transparent model gives people affected by its predictions a chance for recourse.
 - Domain experts can perform a "face validity" check to validate that the model aligns with their intuitive understanding.
- With interpretability in mind, simpler models like logistic regression are often preferable, as they offer direct insight into how predictions are made. Decision trees and certain implementations of random forests also allow for interpretability but at a moderate complexity increase.
- **Model Expressiveness:** This aspect considers how well the model can distinguish between positive and negative outcomes, particularly across diverse groups. More expressive models, like random forests or deep neural networks, may capture complex patterns that logistic regression cannot. However, a more expressive model might also lead to overfitting on smaller datasets or disparities in performance across groups.

Additional considerations:

Training time is a practical factor, impacting how quickly we can iterate on and refine the model. Familiarity with a model can also ease fine-tuning and troubleshooting. Carbon footprint is increasingly important, especially for larger models like neural networks, as these consume significant computational resources and have environmental impacts.

Recommendation: Given these considerations, a logistic regression or random forest might be a good starting point for balancing interpretability with reasonable expressiveness.

✓ Decision point: Model type

We will use a logistic regression model. Our reasoning:

- **Interpretability.** Logistic models over a small number of variables (as used here) are highly interpretable in the sense that stakeholders can simulate them easily.
- **Model expressiveness.** Logistic regression predictions are described by a linear weighting of the feature values. The concern might be that this is too simple.

Prepare training and test datasets

As we mentioned in the task definition, our target variable is **readmission within 30 days**, and our sensitive feature for the purposes of fairness assessment is **race**.

```
target_variable = "readmit_30_days"
demographic = ["race", "gender"]
sensitive = ["race"]
```

```
Y, A = df.loc[:, target_variable], df.loc[:, sensitive]
```

We next drop the features that we don't want to use in our model and expand the categorical features into binary indicators ("dummies").

```
X = pd.get_dummies(df.drop(columns=[
    "race",
    "race_all",
    "discharge_disposition_id",
    "readmitted",
    "readmit_binary",
    "readmit_30_days"
]))
```

```
X.head() # sanity check
```

	time_in_hospital	num_lab_procedures	num_procedures	num_medications	number_diagnoses	medicare	medicaid	had_emergency	has
0	1	41	0	1	1	False	False	False	
1	3	59	0	18	9	False	False	False	
2	2	11	5	13	6	False	False	False	
3	2	44	1	16	7	False	False	False	
4	1	51	0	8	5	False	False	False	

5 rows × 44 columns

We split our data into a training and test portion. The test portion will be used to evaluate our performance metric (i.e., balanced accuracy), but also for fairness assessment. The split is half/half for training and test to ensure that we have sufficient sample sizes for fairness assessment.

```
random_seed = 123
np.random.seed(random_seed)
```

```
X_train, X_test, Y_train, Y_test, A_train, A_test, df_train, df_test = train_test_split(
    X,
    Y,
    A,
    df,
    test_size=0.50,
    stratify=Y,
    random_state=random_seed
)
```

Our performance metric is **balanced accuracy**, so for the purposes of training (but not evaluation!) we will resample the data set, so that it has the same number of positive and negative examples.

Because we are downsampling the number of negative examples, we create a training dataset with a significantly lower number of data points. For more complex machine learning models, this lower number of training data points may affect the model's accuracy.

```
def resample_dataset(X_train, Y_train, A_train):

    negative_ids = Y_train[Y_train == 0].index
    positive_ids = Y_train[Y_train == 1].index
    balanced_ids = positive_ids.union(np.random.choice(a=negative_ids, size=len(positive_ids)))

    X_train = X_train.loc[balanced_ids, :]
    Y_train = Y_train.loc[balanced_ids]
    A_train = A_train.loc[balanced_ids, :]
    return X_train, Y_train, A_train
```

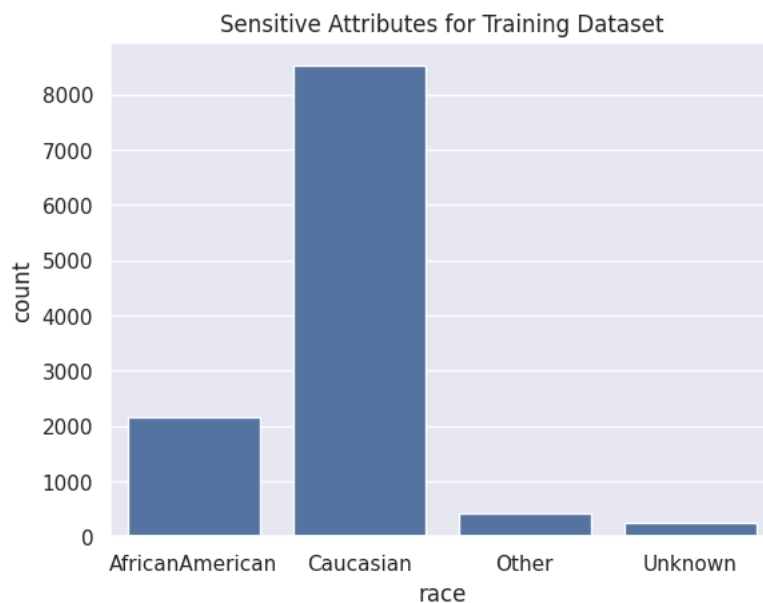
```
X_train_bal, Y_train_bal, A_train_bal = resample_dataset(X_train, Y_train, A_train)
```

Save descriptive statistics of training and test data

We next evaluate the descriptive statistics of the training dataset. These can be provided as part of *model cards* to document our training.

```
sns.countplot(x="race", data=A_train_bal)  
plt.title("Sensitive Attributes for Training Dataset")
```

```
Text(0.5, 1.0, 'Sensitive Attributes for Training Dataset')
```



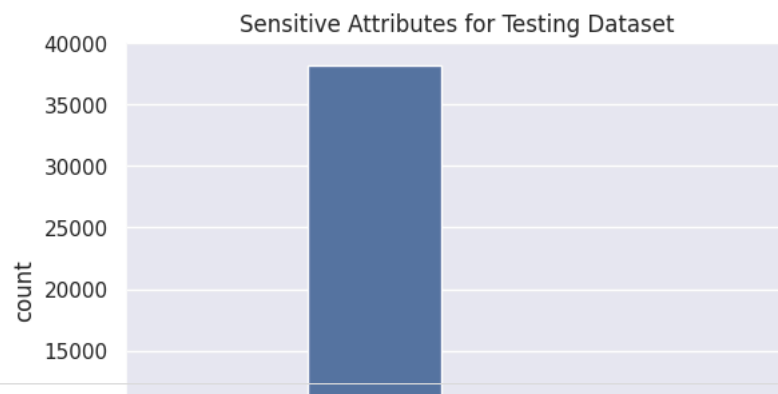
```
sns.countplot(x=Y_train_bal)  
plt.title("Target Label Histogram for Training Dataset")
```

```
Text(0.5, 1.0, 'Target Label Histogram for Training Dataset')
```



```
sns.countplot(x="race", data=A_test)  
plt.title("Sensitive Attributes for Testing Dataset")
```

Text(0.5, 1.0, 'Sensitive Attributes for Testing Dataset')



```
sns.countplot(x=Y_test)
plt.title("Target Label Histogram for Test Dataset")
```

Text(0.5, 1.0, 'Target Label Histogram for Test Dataset')

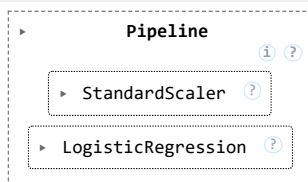


✓ Train the model

We train a logistic regression model and save its predictions on test data for analysis.

```
unmitigated_pipeline = Pipeline(steps=[
    ("preprocessing", StandardScaler()),
    ("logistic_regression", LogisticRegression(max_iter=1000))
])
```

```
unmitigated_pipeline.fit(X_train_bal, Y_train_bal)
```



```
Y_pred_proba = unmitigated_pipeline.predict_proba(X_test)[: ,1]
Y_pred = unmitigated_pipeline.predict(X_test)
```

```
Y_pred
```

```
array([0, 1, 0, ..., 1, 0, 0])
```

Check model performance on test data.

```
# Show balanced accuracy rate of the 0/1 predictions
```

```
balanced_accuracy_score(Y_test, Y_pred)
```

```
np.float64(0.5936067117421118)
```

As we see, the performance of the model is above the performance of a coin flip, albeit it is quite far from a great classifier.

Balanced accuracy measures how well a classifier does on each class equally, by averaging the recall for every class.

$1/2(TPR+TNR)$

✓ Fairness assessment

✓ Measuring fairness-related harms

The goal of fairness assessment is to answer the question: *Which groups of people may be disproportionately negatively impacted by an AI system and in what ways?*

The steps of the assesment are as follows:

1. Identify harms
2. Identify the groups that might be harmed
3. Quantify harms
4. Compare quantified harms across the groups

We next examine these four steps in more detail.

1. Identify harms

For example, in a system for screening job applications, qualified candidates that are automatically rejected experience an allocation harm. In a speech-to-text transcription system, high error rates constitute harm in the quality of service.

In the health care scenario, the patients that would benefit from a care management program, but are not recommended for it experience a harm. In the context of the classification scenario these are **FALSE NEGATIVES**.

2. Identify the groups that might be harmed

In most applications, we consider demographic groups including historically marginalized groups (e.g., based on gender, race, ethnicity).

It is also important to consider group intersections, for example, in addition to considering groups according to gender and groups according to race, it is also important to consider their intersections (e.g., Black women, Latinx nonbinary people, etc.).

In the health care scenario, based on the previous work, we focus on groups defined by **race**.

3. Quantify harms

Define metrics that quantify harms or benefits:

- In job screening scenario, we need to quantify the number of candidates that are classified as "negative" (not recommended for the job), but whose true label is "positive" (they are qualified). One possible metric is the **false negative rate**: fraction of qualified candidates that are screened out.
- In speech-to-text scenario, the harm could be measured by **word error rate**, number of mistakes in a transcript divided by the overall number of words.
- **In the health care scenario**, we could consider two metrics for quantifying harms / benefits:
 - **false negative rate**: fraction of patients that are readmitted within 30 days, but that are not recommended for the care management program; this quantifies harm ($FN/(FN+TP)$)
 - **selection rate**: overall fraction of patients that are recommended for the care management program (regardless of whether they are readmitted within 30 days or no); this quantifies benefit; here the assumption is that all patients benefit similarly from the extra care. ($TP+FP/N$)

There are several reasons for including selection rate in addition to false negative rate. We would like to monitor how the benefits are allocated, focusing on groups that might be disadvantaged. Another reason is to get extra robustness in our assesement, because our measure (i.e., readmission within 30 days) is only an imperfect measure of our construct (who is most likely to benefit from the care management program). The auxiliary metrics, like selection rate, may alert us to large disparities in how the benefit is allocated, and allow us to catch issues that we might have missed.

4. Compare quantified harms across the groups

The workhorse of fairness assessment are *disaggregated metrics*, which are **metrics evaluated on slices of data**. For example, to measure harms due to errors, we would begin by evaluating the errors on each slice of the data that corresponds to a group we identified in Step 2. If some of the groups are seeing much larger errors than other groups, we would flag this as a fairness harm.

To summarize the disparities in errors (or other metrics), we may want to report quantities such as the **difference** or **ratio** of the metric values between the best and the worst slice. In settings where the goal is to guarantee certain minimum quality of service (such as speech recognition), it is also meaningful to report the **worst performance** across all considered groups.

For example, when comparing false negative rate across groups defined by race, we may summarize our findings with a table like the following:

	false negative rate (FNR)
AfricanAmerican	0.43
Caucasian	0.44
Other	0.52
Unknown	0.67
<i>largest difference</i>	0.24 (best is 0.0)
<i>smallest ratio</i>	0.64 (best is 1.0)
<i>maximum (worst-case) FNR</i>	0.67

Fairness assessment with MetricFrame

Fairlearn provides the data structure called `MetricFrame` to enable evaluation of disaggregated metrics. We will show how to use a `MetricFrame` object to assess the trained `LogisticRegression` classifier for potential fairness-related harms.

```
# In its simplest form MetricFrame takes four arguments:
#   metric_function with signature metric_function(y_true, y_pred)
#   y_true: array of labels
#   y_pred: array of predictions
#   sensitive_features: array of sensitive feature values

mf1 = MetricFrame(metrics=false_negative_rate,
                  y_true=Y_test,
                  y_pred=Y_pred,
                  sensitive_features=df_test['race'])

# The disaggregated metrics are stored in a pandas Series mf1.by_group:

mf1.by_group
```

	false_negative_rate
race	
AfricanAmerican	0.409
Caucasian	0.439
Other	0.516
Unknown	0.628

dtype: float64

```
# The largest difference, smallest ratio and worst-case performance are accessed as
#   mf1.difference(), mf1.ratio(), mf1.group_max()

print(f"difference: {mf1.difference():.3}\n"
      f"ratio: {mf1.ratio():.3}\n"
      f"max across groups: {mf1.group_max():.3}")

difference: 0.219
ratio: 0.651
max across groups: 0.628
```

```
# You can also evaluate multiple metrics by providing a dictionary

metrics_dict = {
    "selection_rate": selection_rate,
    "false_negative_rate": false_negative_rate,
    "balanced_accuracy": balanced_accuracy_score,
```

```

}

metricframe_unmitigated = MetricFrame(metrics=metrics_dict,
                                       y_true=Y_test,
                                       y_pred=Y_pred,
                                       sensitive_features=df_test['race'])

# The disaggregated metrics are then stored in a pandas DataFrame:

metricframe_unmitigated.by_group

```

	selection_rate	false_negative_rate	balanced_accuracy
race			
AfricanAmerican	0.413	0.409	0.600
Caucasian	0.397	0.439	0.593
Other	0.343	0.516	0.579
Unknown	0.238	0.628	0.573

```

# The largest difference across the groups:

metricframe_unmitigated.difference()

```

```

      0
selection_rate    0.174
false_negative_rate 0.219
balanced_accuracy 0.027

dtype: float64

```

✓ YOUR TASK: Generate a similar table for "ratio" instead of "difference"

```

# Use ratio:
metricframe_unmitigated.ratio()

```

```

      0
selection_rate    0.578
false_negative_rate 0.651
balanced_accuracy 0.955

dtype: float64

```

```

# You can view all of them in the same table:

```

```

pd.DataFrame({'difference': metricframe_unmitigated.difference(),
              'ratio': metricframe_unmitigated.ratio(),
              'group_min': metricframe_unmitigated.group_min(),
              'group_max': metricframe_unmitigated.group_max()}).T

```

	selection_rate	false_negative_rate	balanced_accuracy
difference	0.174	0.219	0.027
ratio	0.578	0.651	0.955
group_min	0.238	0.409	0.573
group_max	0.413	0.628	0.600

```

# You can also easily plot all the metrics

```

```

metricframe_unmitigated.by_group.plot.bar(subplots=True, layout= [1,3], figsize=(12, 4),
                                           legend=False, rot=-45, position=1.5);

```



We observe disparity, even though we did not include race in our model. There's a variety of reasons why such disparities may occur. It could be due to representational issues (i.e., not enough instances per group), or because the feature distribution itself differs across groups (i.e., different relationship between features and target variable). Real-world applications often exhibit both kinds of issues at the same time.

✓ Mitigating fairness-related harms in ML models

We have found that the logistic regression predictor leads to a large difference in false negative rates between the groups. We next look at **algorithmic mitigation strategies** of this fairness issue (and similar ones).

Note that while we currently focus on the training stage of the AI lifecycle, mitigation should not be limited to this stage. In fact, we have already discussed mitigation strategies at the task definition stage (e.g., checking for construct validity) and data collection stage (e.g., collecting more data).

Within the model training stage, mitigation may occur at different steps relative to model training:

- **Preprocessing:** A mitigation algorithm is applied to transform the input data to the training algorithm; for example, some strategies seek to remove the dependence between the input features and sensitive features.
- **At training time:** The model is trained by an (optimization) algorithm that seeks to satisfy fairness constraints.
- **Postprocessing:** The output of a trained model is transformed to mitigate fairness issues; for example, the predicted probability of readmission is thresholded according to a group-specific threshold.

We will explore a postprocessing approach which is in fact a **meta-algorithm** in the sense that it acts as a wrapper around *any* standard (fairness-unaware) machine learning algorithms. This makes it quite versatile in practice.

✓ Postprocessing with ThresholdOptimizer

Postprocessing techniques are a class of unfairness-mitigation algorithms that take an already trained model and a dataset as an input and seek to fit a transformation function to model's outputs to satisfy some (group) fairness constraint(s). They might be the only feasible unfairness mitigation approach when developers cannot influence training of the model, due to practical reasons or due to security or privacy.

Here we use the `ThresholdOptimizer` algorithm from Fairlearn, which follows the approach of [Hardt, Price, and Srebro \(2016\)](#).

`ThresholdOptimizer` takes in an existing machine learning model whose predictions act as a scoring function and identifies a separate threshold for each group in order to optimize some specified objective metric (such as **balanced accuracy**) subject to specified fairness constraints (such as **false negative rate parity**). Thus, the resulting classifier is just a suitably thresholded version of the underlying machine learning model.

The constraint **false negative rate parity** requires that all the groups have equal values of false negative rate.

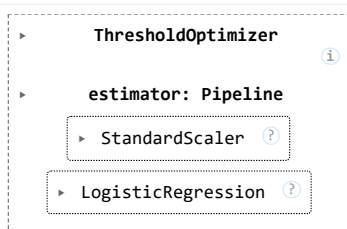
To instantiate our `ThresholdOptimizer`, we need:

- An existing `estimator` that we wish to threshold.
- The fairness `constraints` we want to satisfy.
- The `objective` metric we want to maximize.

```
# Instantiate ThresholdOptimizer with the logistic regression estimator
postprocess_est = ThresholdOptimizer(
    estimator=unmitigated_pipeline,
    constraints="false_negative_rate_parity",
    objective="balanced_accuracy_score",
    prefit=True,
    predict_method='predict_proba'
)
```

In order to use the `ThresholdOptimizer`, we need access to the sensitive features **both during training time and once it's deployed**.

```
postprocess_est.fit(X_train_bal, Y_train_bal, sensitive_features=A_train_bal)
```



```
# Record and evaluate the output of the trained ThresholdOptimizer on test data
```

```
Y_pred_postprocess = postprocess_est.predict(X_test, sensitive_features=A_test)
metricframe_postprocess = MetricFrame(
    metrics=metrics_dict,
    y_true=Y_test,
    y_pred=Y_pred_postprocess,
    sensitive_features=A_test
)
```

We can now inspect how the metric values differ between the postprocessed model and the unmitigated model:

```
pd.concat([metricframe_unmitigated.by_group,
           metricframe_postprocess.by_group],
          keys=['Unmitigated', 'ThresholdOptimizer'],
          axis=1)
```

	Unmitigated			ThresholdOptimizer		
	selection_rate	false_negative_rate	balanced_accuracy	selection_rate	false_negative_rate	balanced_accuracy
race						
AfricanAmerican	0.413	0.409	0.600	0.388	0.447	0.593
Caucasian	0.397	0.439	0.593	0.388	0.446	0.593
Other	0.343	0.516	0.579	0.338	0.533	0.572
Unknown	0.238	0.628	0.573	0.447	0.457	0.552

We next zoom in on differences between the largest and the smallest metric values:

```
pd.concat([metricframe_unmitigated.difference(),
           metricframe_postprocess.difference()],
          keys=['Unmitigated: difference', 'ThresholdOptimizer: difference'],
          axis=1).T
```

	selection_rate	false_negative_rate	balanced_accuracy	
Unmitigated: difference	0.174	0.219	0.027	
ThresholdOptimizer: difference	0.110	0.087	0.041	

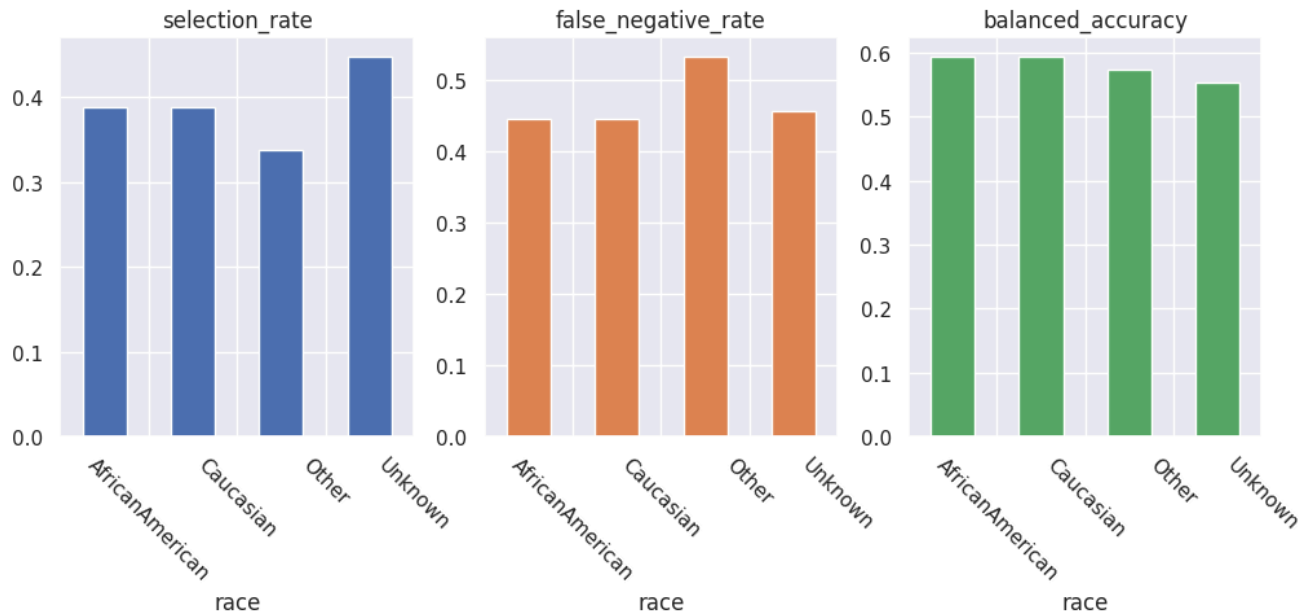
As we see, `ThresholdOptimizer` was able to substantially decrease the difference between the values of false negative rate.

The ThresholdOptimizer adjusts predictions from the pretrained logistic regression model by setting unique cut-off points / "thresholds" / for each group based on the sensitive feature (here - race) without altering the model itself. Normally, predictions with probabilities (Y_pred_proba) over 0.5 are labeled as 1 (re-admission within 30 days), while those below are 0. With the optimizer, each group (i.e. different race categories) can have its own threshold to meet our fairness constraint (false negative rate parity). For example, the threshold for the "Unknown" group may be lowered to 0.45, so predictions at 0.45 are labeled as 1 for that group. This process uses the race variable during testing to apply the right threshold per group.

Finally, we plot the disaggregated statistics:

```
metricframe_postprocess.by_group.plot.bar(subplots=True, layout=[1,3], figsize=(12, 4), legend=False, rot=-45, position=1.5)
```

```
array([[<Axes: title={'center': 'selection_rate'}, xlabel='race'>,
        <Axes: title={'center': 'false_negative_rate'}, xlabel='race'>,
        <Axes: title={'center': 'balanced_accuracy'}, xlabel='race'>]],
      dtype=object)
```



Model performance - by group

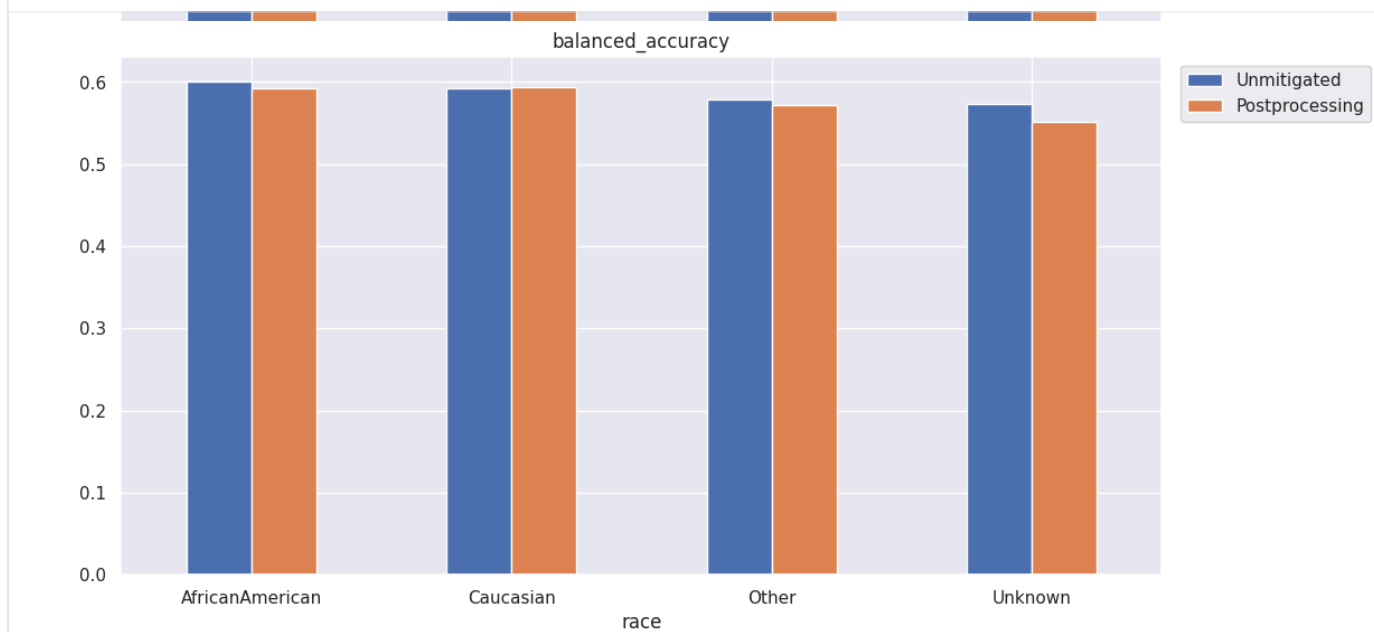
```
def plot_technique_comparison(mf_dict, metric):
    """
    Plots a specified metric for a given dictionary of MetricFrames.
    """
    mf_dict = {k:v.by_group[metric] for (k,v) in mf_dict.items()}
    comparison_df = pd.DataFrame.from_dict(mf_dict)
    comparison_df.plot.bar(figsize=(12, 6), legend=False)
    plt.title(metric)
    plt.xticks(rotation=0, ha='center');
    plt.legend(bbox_to_anchor=(1.01,1), loc='upper left')
```

```
test_dict = {
    "Unmitigated": metricframe_unmitigated,
    "Postprocessing": metricframe_postprocess
}
```

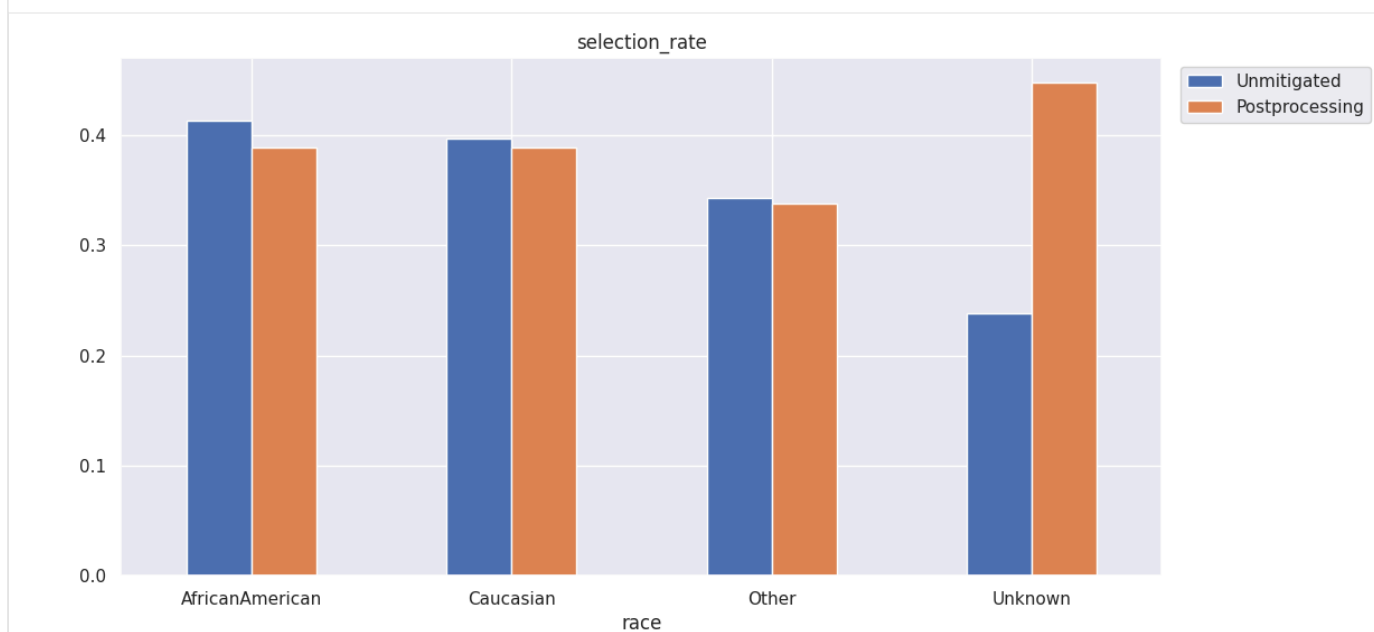
```
plot_technique_comparison(test_dict, "false_negative_rate")
```



```
plot_technique_comparison(test_dict, "balanced_accuracy")
```



```
plot_technique_comparison(test_dict, "selection_rate")
```



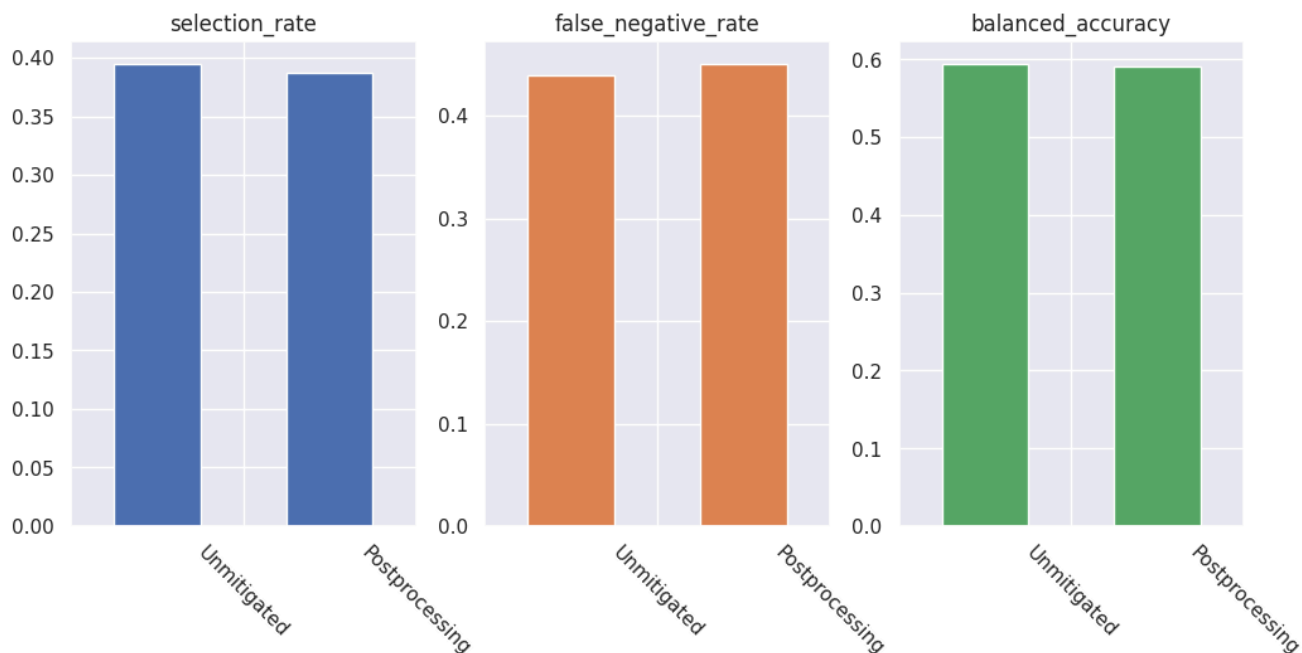
Model performance - overall

```
overall_df = pd.DataFrame.from_dict({
    "Unmitigated": metricframe_unmitigated.overall,
    "Postprocessing": metricframe_postprocess.overall
})
```

overall_df.T

	selection_rate	false_negative_rate	balanced_accuracy
Unmitigated	0.394	0.439	0.594
Postprocessing	0.387	0.450	0.591

```
overall_df.transpose().plot.bar(subplots=True, layout= [1,3], figsize=(12, 5), legend=False, rot=-45, position=1.5);
```

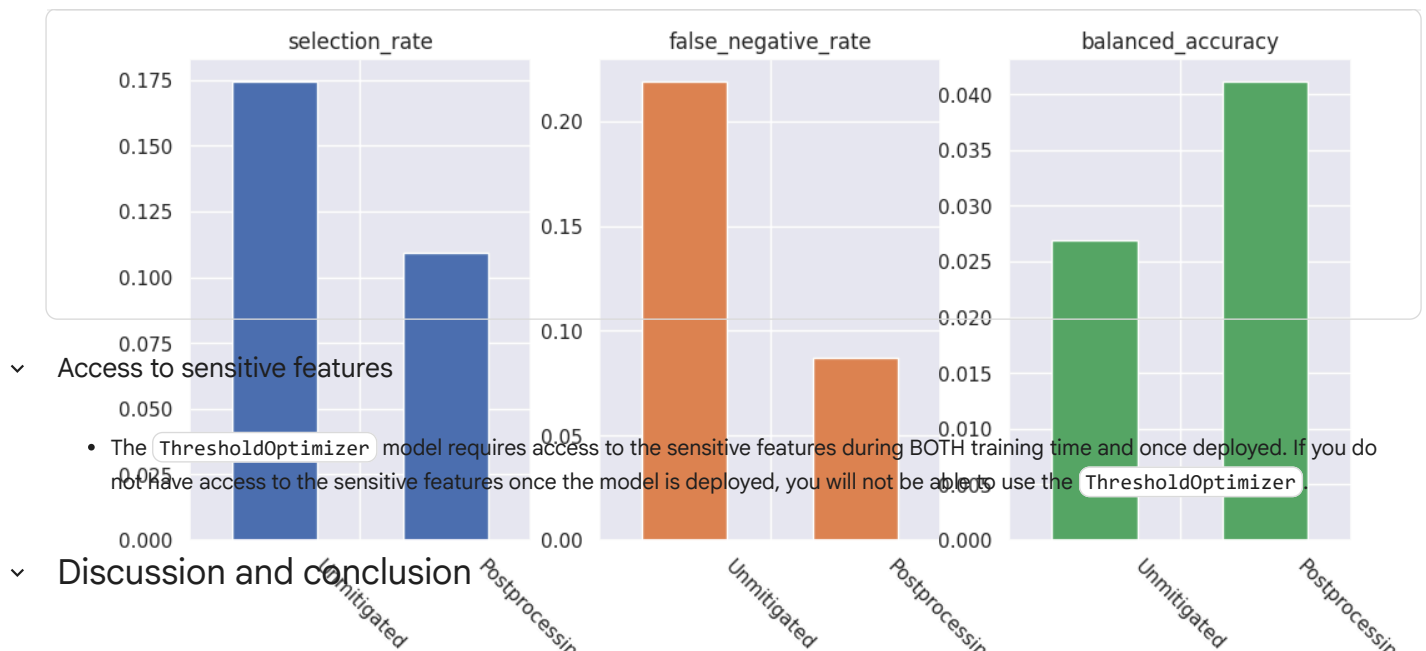


```
difference_df = pd.DataFrame.from_dict({
    "Unmitigated": metricframe_unmitigated.difference(),
    "Postprocessing": metricframe_postprocess.difference()
})
```

difference_df.T

	selection_rate	false_negative_rate	balanced_accuracy
Unmitigated	0.174	0.219	0.027
Postprocessing	0.110	0.087	0.041

```
difference_df.T.plot.bar(subplots=True, layout= [1,3], figsize=(12, 5), legend=False, rot=-45, position=1.5);
```



In this tutorial, we walked through a healthcare scenario across each stage of the AI lifecycle, except model deployment. We explored how fairness-related harms can emerge at each stage - from defining the task and collecting data to training and evaluating the model. Along the way, we used tools and practices like Fairlearn to address these concerns.

After deployment, it's essential to keep monitoring key metrics to detect performance variations or potential fairness issues. As more is learned about how the model is used, you may need to refine fairness metrics, update the model, incorporate new sensitive features, revisit the task definition, or gather additional data.

While we used a range of software tools, it's important to remember that fairness is a sociotechnical challenge. Technical fixes alone aren't enough; they need to be supported by broader processes, practices, and often, regulatory and organizational changes.

If you're interested in learning more about fairness in AI or contributing to Fairlearn, you're invited to join the community, where contributors bring diverse expertise and perspectives to this work.

Further resources can also be found [on the FAIRLEARN website](https://fairlearn.org/).